COMPUTER SCIENCE + COMPUTER ENGINEERING

Start the next package

Expanded Volume 1 / continuation guide

Where to continue, what to open, how to run the examples, and how to distinguish your work from the reference solutions.

20 lessons · 5 modules · 1 completed first-game milestone

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Start the next package

Package 02 is the expanded first teaching volume

The first ZIP supplied the full-program roadmap and compact primers for all ten volumes. This package expands **Volume 1: Foundations and Your First Game**, preserving Modules 01-05 and lesson IDs 01.1-05.4. Package number 02 is not Volume 02: the mathematics volume is the next expansion in the program sequence, not part of this ZIP.

Included: 20 expanded lessons and runnable examples; 20 practical briefs; 16 code-exercise starter/reference pairs; four evidence-based labs; 100 lesson quiz questions; 25 module-review questions; 50 additional exercises; five module projects; a full Gate A specification; separate worked answers; a completed reference game; and intentionally unfinished student capstone scaffolding.

The 100 lesson questions retain the original three questions per lesson and add two application questions. The 25 module-review questions are additional. Thus 60 quiz questions are retained from the starter and 65 are new; this is not a claim that all 125 are new. The 50 exercises are separate from quiz counts.

Choose the correct starting lesson

If the first-day work in the original package is complete, begin at **01.3: Trace before running** after checking the new application questions for 01.1 and 01.2. Otherwise start at 01.1. Experienced learners may move faster by demonstrating the required work, not by skipping the practical evidence automatically.

The early 01.1, 01.3 and 01.4 lessons are paper-first. Their optional function scaffolds use syntax introduced in Module 02: return to those files after Lesson 02.4. Do not mistake the presence of a function starter for an undeclared first-day prerequisite.

Open the material

Extract the ZIP. Open `STUDENT_HANDBOOK.pdf` for the main course or `COURSE.html` for the searchable browser edition. Keep `ANSWER_KEY.pdf` closed until after an independent attempt. Use `LAB_WORKBOOK.pdf` for the practical tasks, additional exercises and module-review sheets. Use `CAPSTONE_SPEC.pdf` when assembling the final game.

All examples and tests run locally. The browser edition needs no server or account. External reading links require internet access. Local browser progress is self-recorded, can vary by browser/file location, and should be exported regularly. Nothing in the course sends your progress to a server.

Prepare the terminal

Open a terminal at the extracted `Game_Engineering_Package_02` folder, where examples/, tools/, labs/ and capstone/ are visible. The commands below use the macOS/Linux launcher. On Windows, replace `python3` with a verified `py -3` or `python` launcher. Do not include `$`, `PS>` or `>>>` prompt characters.

```
python3 --version
python3 tools/environment_check.py
python3 examples/example_01_3.py
```

The runtime code targets Python 3.11 or later and uses the standard library. It was executed with Python 3.13.5 in Linux; Windows/macOS and other Python versions are not claimed as physically tested. Use the official Python setup source [S03] when installation is needed. Git is needed at 04.3, not to begin. No game engine, GPU toolkit, cloud account or third-party Python package is needed for these supplied examples.

Complete one lesson

Read the objective and prerequisite. Write a prediction. Run and explain the worked example. Complete the practical brief and changed scenario. Answer all five lesson questions before checking the key. Keep the original core questions numbered 1-3 distinct from the new application questions numbered 4-5.

Use 0 points for incorrect/absent, 1 for partial or unexplained, and 2 for correct with reasoning. Suggested pass: **at least 5/6 on questions 1-3 and at least 8/10 overall**, plus the assigned study, practical pass condition and a changed-input explanation. A short quiz alone is not evidence of mastery.

At each module end, complete ten additional exercises, its five-question review, and the integration project. The suggested module-review threshold is 8/10 with explanations. The project rubric requires 80/100 and all essential checks, regardless of a high total in optional areas. These are formative self-study rules, not validated university examinations.

Know which directory you are testing

```
# Check YOUR edited function after Lesson 02.4:
python3 tools/check_labs.py --lesson 02.4

# Check every supplied REFERENCE function:
python3 tools/check_labs.py --reference

# Run the completed REFERENCE game:
python3 capstone/reference/play.py

# Check the completed REFERENCE game:
python3 tools/check_game.py
```

Student function files raise `NotImplementedError` until completed. The student capstone test deliberately fails until replaced with real learner-authored tests. These expected failures are not failed installation checks. Reference-mode success proves only that the supplied reference passed those checks; it does not test your changes.

The four evidence-based labs are 01.2 (environment), 04.3 (Git), 04.4 (review) and 05.4 (release). They have worksheets and manual evidence requirements instead of a function checker. Early paper-first labs have optional code extensions after 02.4.

A practical first continuation session

For someone who finished the original first two lessons: trace the 01.3 example before running, complete its three assignment sequences, and answer its five questions. Then begin the unit table in 01.4. Stop at a saved working checkpoint rather than racing through the next module. Do not start by opening the whole completed reference game and trying to memorize it.

The original guide reserves 220-340 planning hours for Volume 1, including deeper exercises and the project. This package expands that volume's teaching and evidence; it does not add a second independent block of 220-340 hours. Actual time depends on preparation, practice and review. Completion is determined by demonstrated work, not pages read.

Where the student should end up

An independently built, explainable terminal game with pure updates, collision rules, win/loss and restart, validated save/load, deterministic replay, learner-owned tests and a clean handoff. The package is a self-study course, not an accredited degree or a claim of professional qualification.